

Calling the Shot

Operational Mood, Deferred Formalization, and the Distributed Prompt
Program

Watson Hartsoe

WP-20260817-2035-7696ff6a41

2026-08-17

AI-AUGMENTED WORKING PAPER. The exact assembly prompt, source map, making history, and rebuild path are included in the accompanying SLIPCASE package. This manuscript is a current

wager over the compiled evidence, not itself archival evidence.



Abstract

Natural-language programming is often framed as a contest between prompts and code: either ordinary language becomes precise enough to replace programs, or prompts remain an imprecise interface wrapped around conventional software. Current agent practice suggests a different unit. A request can remain linguistically incomplete while a system progressively formalizes only the portions that have become ready for execution. Requirements can be elicited before commitment; action mode can be made explicit; rules can be localized beside tools; output shapes can be typed by schemas; dependencies can be declared as workflows; specifications can be linted before generation; deterministic subregions can compile into temporary code; proposed actions can be reviewed before commitment; execution traces can rewrite future instructions; and revoked constraints can be represented as tombstones. I call this pattern *deferred formalization*. The operative program is not the original prompt, nor the generated code, nor the runtime harness in isolation. It is a distributed and temporal control structure assembled across language, schemas, tools, state, verification, and execution boundaries. This paper reconstructs that structure from current 2025–2026 documentation and research on agentic systems and argues that the central design question is no longer whether natural language is precise enough to program. It is where precision should be introduced, at what scope, and at which moment a provisional intention should become a committed state transition.

1 The prompt is becoming the wrong object

In 2022, prompt craft could plausibly be described as a peculiar linguistic skill: say what you want in a way that survives a statistical interpreter. That description still matters, but it no longer captures the operational surface of an agent that can inspect files, call APIs, edit repositories, send messages, and mutate external state. The contemporary problem is not only whether the model understands the sentence. It is whether the sentence authorizes action, which action surface receives the rule, what representation the action must take, what state must exist before it is legal, and whether the proposed transition should actually be committed.

The current field therefore makes a useful distinction between a *request* and an *operative specification*. REprompt explicitly treats user prompts in software development as carriers of requirements and maps requirements engineering stages—elicitation, analysis, specification, and validation—onto prompt refinement [7]. The first utterance can be raw requirements rather than finished program text. The practical implication is important: natural-language programming need not require the user to fully formalize a problem before computation starts. Formalization can be delayed until interaction reveals what requires precision.

This already changes the familiar argument about whether prompts are “real programming.” If a request is provisional, then comparing it directly with source code compares different stages of a process. A better comparison begins with the entire chain by which an initially incomplete intention becomes executable.

2 Operational mood: language that authorizes state change

Current prompting guidance distinguishes between language that solicits suggestions and language that directs implementation. Anthropic’s documentation notes that a request phrased as a request

for suggested changes can elicit suggestions rather than the changes themselves, and advises greater explicitness when action is intended [1]. This is more than a style recommendation. It identifies an operational variable not captured by propositional content alone.

Consider three utterances: “analyze the configuration,” “tell me how to change the configuration,” and “change the configuration.” They may concern the same desired end state, yet they license different relations between model and environment. The first keeps the system in observation. The second licenses recommendation. The third can authorize mutation. The distinction resembles grammatical mood, but with computational consequence. I call it *operational mood*: the part of an instruction that specifies whether the agent should observe, explain, recommend, prepare, execute, or verify.

Operational mood is necessary because an agent can fail in two opposite directions. Underexecution leaves the user with commentary where a requested action was possible. Overexecution crosses into state change when the user intended only deliberation. Strong prompt practice therefore does not simply make imperatives stronger. It makes authorization legible.

3 The distributed prompt program

Once action is authorized, a second change appears: the specification is no longer located in one prompt. OpenAI’s function-calling guidance recommends treating function descriptions as durable interface contracts and placing invocation criteria and argument rules close to the tool they govern [2]. Structured Outputs separately constrain the legal shape of model output using schemas [3]. Multi-tool examples encode ordering relationships among observations, eligibility checks, mutations, and notifications. These mechanisms distribute the operative specification across several sites.

The useful analogy is scope. Some rules are global invariants; others make sense only for a tool, an argument, a state, a project, or a turn. A rule such as “never fabricate an identifier” may be universal. “Only call refund after delivery and eligibility are confirmed” belongs to the operational neighborhood of the refund path. “Return exactly these fields” may belong in a schema rather than prose. The prompt program is therefore distributed across global language, local tool contracts, structural types, and cross-tool dependencies.

Schemas add a further distinction. Structural validity and semantic validity are different. A model can return an object that perfectly satisfies a JSON schema while selecting the wrong operation or encoding an impossible relationship. This is why the next stage resembles static analysis. SCMAPR, in text-to-video generation, routes prompts by scenario, synthesizes rewriting policies, and performs structured semantic verification that can trigger another revision before the expensive generation step [8]. The broader opportunity is prompt linting: detect contradiction, missing entities, impossible temporal order, or unbound references before execution while preserving intentional underdetermination as creative freedom rather than treating every open variable as an error.

4 Deferred formalization: when language becomes code

Programmatic Tool Calling makes the movement from language to formal procedure explicit. OpenAI documents a mode in which models compose JavaScript that orchestrates tool calls, loops, conditions, parallel operations, and intermediate results [4]. The user does not have to supply that code. A model can identify a bounded, predictable region of the task, generate the temporary control flow needed to execute it, and then return to model-mediated judgment.

This suggests a criterion more useful than the language-versus-code distinction. The boundary is between *judgment* and *procedure*. Where fresh semantic interpretation is still required, natural language and model reasoning remain useful. Where inputs, operations, and branching have stabilized, a region can be compiled into deterministic control flow. Formalization is deferred until the local uncertainty has fallen far enough.

But the movement is bidirectional. Natural-Language Agent Harnesses asks whether high-level harness logic normally buried in controller code can instead be externalized as an executable natural-language object interpreted by a shared runtime through explicit contracts and durable artifacts [9]. The same technical field therefore contains both directions: language can compile into code, while control code can migrate back into editable language. There is no terminal representation toward which all systems converge. The boundary itself is an engineering variable.

This is the strongest reason not to identify the prompt with the program. Execution belongs to a coupled system. Natural language becomes executable only relative to a runtime that gives terms operational meaning: which tools exist, what state persists, how actions are represented, what outputs are validated, and what happens when execution fails.

5 Call the shot before firing it

Tool use makes consequence unavoidable. A model can now form a proposed action that is neither ordinary speech nor completed action. Reinforced Agent places a reviewer between a primary agent’s provisional tool call and execution [5]. This creates a clean temporal distinction: the system can decide what it wants to do, render that decision as an action-shaped object, inspect it, and only then permit the external state transition.

The useful primitive is a commit boundary. `CALL_SHOT` produces a candidate action. The separate `COMMIT` operation crosses into the world. This separation is especially valuable when actions are public, destructive, financially costly, privacy-sensitive, or difficult to reverse.

Yet adding a gate does not solve steering. SteerBench-Work makes the pre-commit proceed-or-hold decision a benchmark object and reports an asymmetric failure pattern [10]: across its evaluated conditions, models wrongly held authorized, evidence-cleared work much more often than they wrongly allowed unsafe work. The hard cases include risks that were real earlier in the scenario but had subsequently been resolved by evidence. The implication is subtle. Safety cannot be represented merely as detection of triggers. A commit gate must reason over the *state* of a trigger: unresolved, cleared, superseded, revoked, or still active.

The action boundary therefore becomes the place where several parts of the distributed prompt program meet: authorization mode, local tool contract, evidence state, policy state, and commit-

ment state.

6 The specification has a history, including negative history

If agent behavior changes through experience, the operative specification becomes temporal. SCOPE treats the agent’s prompt as an evolvable parameter and synthesizes guidelines from execution traces [6]. In this design, failure can become a specification delta. A failed trajectory does not merely cause retry; it can author a rule that governs later behavior.

This mechanism fits a practical pattern familiar from interactive prompting. A user generates an artifact, inspects a failure, names the failure precisely, and turns that correction into a constraint. Over time, accepted corrections become invariants. What matters is not one prompt but the genealogy through which constraints acquired force.

However, a genealogy cannot be append-only. Zhu’s recent work on constraint influence shows that revocation does not always reliably take effect in multi-turn dialogue: a withdrawn requirement can remain behaviorally active [11]. The proposed remedy includes a contract ledger, explicit tombstones for revoked constraints, and ahead-of-time compilation of the net active specification. The striking result for natural-language programming is that deletion from visible prose is not equivalent to deletion from behavior.

A mature prompt program therefore needs negative memory. Each constraint requires a lifecycle, not just text: active, revoked, superseded, expired, unresolved. Current behavior should depend on the active specification at time t , not simply on all instructions ever observed before t . This also clarifies the SteerBench-Work problem. A risk trigger that was later cleared is analogous to a constraint whose state changed. If the agent remembers the trigger but not the clearance, history becomes a source of stale control.

7 The emerging operational grammar

The field supports a provisional grammar of shot-calling. It is not source syntax; it is a reconstruction across current practices.

1. **Requirements state.** What is explicit, inferred, unknown, defaulted, discoverable, or blocking?
2. **Operational mood.** Observe, explain, recommend, prepare, execute, or verify?
3. **Scope.** Which rules are global, project-local, state-local, tool-local, argument-local, or turn-local?
4. **Type.** What output shapes are legal?
5. **Dependency.** What must exist before an action becomes legal?
6. **Static verification.** Which contradictions or omissions can be detected before execution?

7. **Compilation boundary.** Which region is deterministic enough to become code?
8. **Provisional action.** What candidate transition is being proposed?
9. **Commit gate.** Should the candidate proceed or hold given current evidence and policy state?
10. **Postcondition.** What consequence must be verified after execution?
11. **Specification delta.** What did this outcome teach that should govern another attempt?
12. **Constraint lifecycle.** Which old rules have ceased to govern?

This grammar explains why “prompt engineering” increasingly feels too small. The decisive work happens not only in word choice but in the arrangement of representations and transitions around the model. The natural-language portion remains important because it is unusually cheap to revise while the problem is still being discovered. Formal representations become valuable later and locally, when the cost of ambiguity exceeds the value of keeping an issue open.

8 Implications for natural-language programming

The central design question changes. Instead of asking whether English can ever be precise enough to replace code, ask: *Where should precision live, and when should it become binding?*

This formulation supports three implications.

First, natural-language programming is likely to remain hybrid. The strongest current systems do not eliminate schemas, tools, code, or runtime contracts. They use language to coordinate when and where those formal structures should appear.

Second, the specification is distributed. If a researcher archives only the visible user prompt, they may lose the actual program: tool descriptions, schemas, harness instructions, reviewer policies, state ledgers, and generated orchestration code may all contribute to behavior. Reproducibility therefore requires preserving the control field, not just the opening utterance.

Third, the specification is genealogical. A long-lived agent accumulates rules because things went wrong, but it must also know why those rules exist, which model or task produced them, and when they cease to apply. The future prompt may look less like a polished paragraph and more like a versioned contract with provenance, tests, scope, and tombstones.

9 Conclusion

The current evidence does not support the claim that prompts have become programs in the ordinary sense. It supports something more interesting. Natural language has become a medium in which computation can begin before the entire specification has been formalized. As the task unfolds, different portions of intent can acquire stronger forms: requirements, scoped rules, schemas, workflows, checked specifications, temporary code, provisional actions, commit decisions,

verified consequences, and remembered corrections. Formalization is not avoided. It is moved to the point where it becomes useful.

The resulting unit is neither prompt nor code. It is a staged control system whose representations can change during execution. Calling the shot means more than naming the desired outcome. It means specifying enough about authority, scope, state, and commitment that the system can cross from language into consequence without pretending that every uncertainty had to be resolved at the beginning.

Evidence boundary

This working paper was assembled from twelve admitted SHOT zettels dated 2026-08-17 and their registered sources. Earlier MJ-* zettels remain referenced as unresolved ancestors in the packaged graph because their exact card payloads were not independently machine-extracted for this checkpoint. The source map in the package distinguishes sourced claims from compiler synthesis.

References

- [1] Anthropic. *Prompting best practices*. Claude Platform documentation, accessed 2026-08-17.
- [2] OpenAI. *o3/o4-mini Function Calling Guide*. OpenAI Cookbook, 2025-05-26.
- [3] OpenAI. *Structured model outputs*. OpenAI API documentation, accessed 2026-08-17.
- [4] OpenAI. *Programmatic Tool Calling*. OpenAI API documentation, accessed 2026-08-17.
- [5] Anh Ta, Junjie Zhu, and Shahin Shayandeh. *Reinforced Agent: Inference-Time Feedback for Tool-Calling Agents*. arXiv:2604.27233, 2026.
- [6] Zehua Pei et al. *SCOPE: Prompt Evolution for Enhancing Agent Effectiveness*. arXiv:2512.15374, 2026 version.
- [7] Junjie Shi et al. *REprompt: Prompt Generation for Intelligent Software Development Guided by Requirements Engineering*. arXiv:2601.16507, 2026.
- [8] Chengyi Yang et al. *SCMAPR: Self-Correcting Multi-Agent Prompt Refinement for Complex-Scenario Text-to-Video Generation*. arXiv:2604.05489, 2026.
- [9] Linyue Pan et al. *Natural-Language Agent Harnesses*. arXiv:2603.25723, 2026.
- [10] Oguz Serdar and Cuneyt Mertayak. *SteerBench-Work: A Benchmark for Agent Steering at Action Boundaries*. arXiv:2608.12654, 2026.
- [11] Haoyuan Zhu. *Dead text or binding clause? Measuring and restoring constraint influence in black-box LLM dialogues*. arXiv:2608.12599, 2026.